

3.4 GENERATING STEERED SEQUENCES

Once a latent corresponding to a specific feature is identified, we can steer the model during inference to increase the likelihood of generating protein sequences that contain that feature. This approach, previously demonstrated in natural language models by (Templeton et al., 2024), is outlined in figure 1.

We begin with a randomly generated amino acid sequence of fixed length. After a forward pass through the model and the encoder layer of the SAE, we modify the target latent z_k by scaling and shifting its value to increase its magnitude (equation 7). We then pass the modified value z_k^* through the decoder layer f_{dec} and add back the original reconstruction error of the embedding x before passing it through the rest of the model (equation 8).

$$z_k^* = a \cdot z_k + b \quad (7)$$

$$x^* = f_{dec}(z_k^*) + x_{err} \quad (8)$$

For each position in the sequence, we randomly sample an amino acid according to the probability distribution predicted by the model under the intervention. We repeat this process starting from the predicted sequence and perform 100 iterations of inference-prediction to refine the sequence. We select the sequence at the iteration where the value of the activation z_k is maximum.

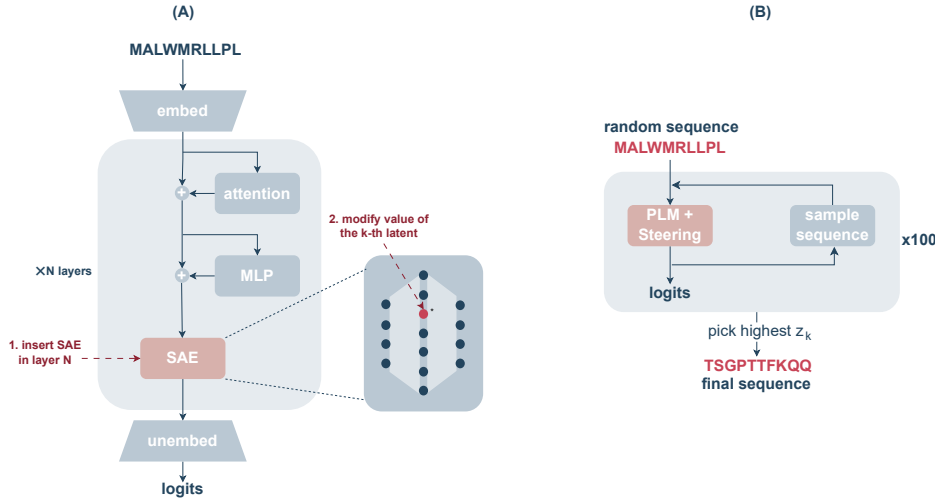


Figure 1: Sequence generation procedure. (A) To steer the model outputs, the base Protein Language Model is modified through the insertion of a sparse autoencoder in the residual stream, at a particular layer. During inference, the value of one of the latents in the autoencoder is modified. (B) Starting from a random sequence, we perform inference with the modified and intervened model, and sample a new sequence from the output logits. We repeat this procedure iteratively a certain number of times (i.e. 100), and at the end we retain the sequence which gives the highest value for the activation of the target latent z_k .

4 RESULTS

4.1 INTERPRETING LATENTS

For the interpretability analysis, we focus on the autoencoder that provides the best trade-off between sparsity and reconstruction quality, as described in section A.3.2 of the appendix. We compute recall and precision for all $[k, \phi]$ pairs, following the methodology outlined in section 3.3, using three increasing thresholds of latent activation. This allows us to assess the robustness of the identified features.

Table 1: Number of latent-feature annotation pairs with a minimum precision/recall of 0.8 for different values of the activation threshold τ_k

τ_k	# Pairs (Precision > 0.8)	# Pairs (Recall > 0.8)	Total
0.01	4	262	266
0.10	8	234	242
1.00	133	61	194
Total (unique)	133	262	395

We find 395 putative $[k, \phi]$ associations, detailed in table 1. Among these, there are latent components associated to different binding sites, cellular regions and motifs like zinc fingers. The complete set of latent - feature associations is available in the supplementary data (see section 5).

We also identify many potential associations with a lower confidence (lower values of precision or recall). To get an idea of how many putative association are found for each value of precision/recall (as well as a combined F1-score) we plot their cumulative distributions in figure 2.

Intuitively, a latent component that perfectly matches an annotation type should exhibit both high precision and recall, resulting in a high F1-score. However, since the model is trained to optimize a masked language modeling loss, the features it learns may not directly align with those in the manually curated dataset. For instance, a latent k might encode a more specific subcategory of a dataset label ϕ , such as identifying the starting amino acid of a helix rather than the entire helix structure (as seen by Adams et al. (2025) on some features). In such cases, the association between k and ϕ would likely have high precision but low recall.

Similarly, a high recall but low precision may indicate that the model has learned a more coarse-grained feature than those defined in the dataset. This is evident in cases such as latent $k = 610$, which activates across various types of zinc fingers, and latent $k = 555$, which responds to alpha-keto acids. To prevent selecting latents with high recall due to trivial reasons – such as activating indiscriminately on all amino acids – we also assess the proportion of times a latent is active on amino acids lacking a given label, denoted as $P(k^+|\phi^-)$, before confirming an association. In section A.3.3, we present examples of the distributions of $P(k^+|\phi^+)$ and $P(k^+|\phi^-)$ for a zinc finger region.

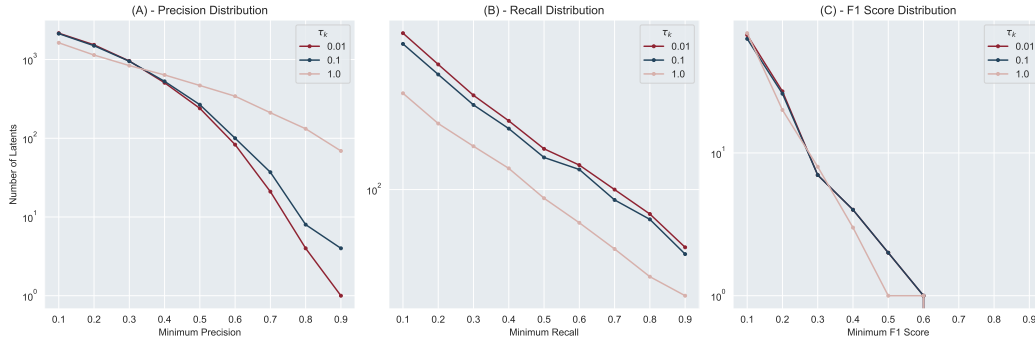


Figure 2: Distribution of the number of latent SAE components that detect a feature with a minimum value of precision, recall and F1-score. Setting a higher value for the activation threshold τ_k significantly increases the precision with which latents detect features, but it decreases the recall.

4.2 SEQUENCE GENERATION

We test the sequence generation procedure with shortlisted latents with a clear association to the zinc finger region annotations (high recall in this specific case). We probe different values of sequence length ($[22, 27, 30, 35, 40, 60]$), of the scaling a ($[2, 5, 10, 20, 30]$) and shift b ($[0.1, 1, 10, 50, 100, 200]$), for a total of 180 combinations, obtaining one sequence for each.

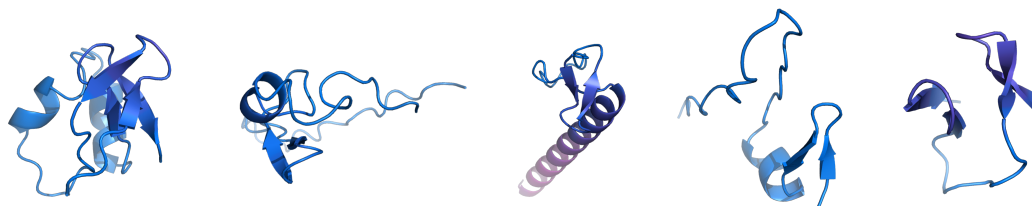


Figure 3: Examples of generated sequences subsequently folded with ESMFold (Lin et al., 2023). The sequences were generated while intervening on the model by increasing the value of latent components associated to the zinc finger motif. With the intervention, the model has a tendency to generate pairs of beta sheets in the vicinity of a helix, as in a typical zinc finger structure.

Using an online tool for automatic annotation of zinc finger regions (Sathyaseelan et al., 2023), we look for known zinc finger motifs and Pfam family matches in our generated sequences. We find 24 matching regions (out of 180 sequences) when we simultaneously intervene on the two most prominent latents (highest recall) displaying a zinc finger association. In contrast, intervening on the most prominent latent produces only 3 matches, while generating sequences from the baseline model or intervening on a random latent or a random pair of latents produces no matches in any case. Among the matched sequences, the highest percent similarity was 48%, with an average sequence similarity of 31%, indicating a good level of diversity among the generated sequences.

While this sequence generation pipeline requires parameter fine-tuning to improve the success rate, the process can be automated by introducing appropriate heuristics to search the parameter space efficiently. To the best of our knowledge, this is the first application of steering with sparse autoencoder features to generate complex protein sequences, extending beyond trivial features like specific amino acids or simple amino acid repeats.

5 DISCUSSION AND CONCLUSIONS

In this study, we demonstrate the potential of sparse autoencoders (SAEs) for interpreting and manipulating the internal representations of protein language models. By training a SAE on the ESM-2 8M parameter model, we identified and interpreted latent features associated with various protein annotations, including transmembrane regions, binding sites, and zinc finger motifs.

We have also demonstrated, for the first time, that these latent components can be leveraged to steer the model towards generating protein sequences with non-trivial structural features, like zinc finger motifs. The results highlight the utility of SAEs in disentangling the complex, polysemantic representations within protein language models, paving the way for more interpretable and controllable sequence generation. This approach not only deepens our understanding of how these models encode biological features but also opens up new possibilities for protein design and engineering. Future work could extend these methods to larger models and a wider range of protein features, further bridging the gap between interpretability and practical applications in computational biology.

CODE & DATA AVAILABILITY

The weights for the trained sparse autoencoder model are available from huggingface at: https://huggingface.co/evillegasgarcia/sae_esm2.6_l3. The code is available from github at: <https://github.com/edithvillegas/plm-sae>. Supplementary data is available from zenodo at <https://doi.org/10.5281/zenodo.14837817>

ACKNOWLEDGMENTS AND DISCLOSURE OF FUNDING

We thank the technical support of the Laboratory of Data Engineering staff and acknowledge the AREA Science Park supercomputing platform ORFEO.